

PythonIntermediateCourse-es015ML

June 28, 2026

1 Curso básico de Python

1.1 Apuntes intermedio

Curso básico de Python. Apuntes intermedio por Marcelo Horacio Fortino. Versión 2.5.2. Junio 2026.

Esta obra está sujeta a la licencia Reconocimiento-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-sa/4.0/>. Puede hallar permisos más allá de los concedidos con esta licencia en <https://fortinux.com>. Sugerencias y comentarios a info@fortinux.com.

Todas las marcas son propiedad de sus respectivos dueños. Python® y PyCon® son marcas registradas de la Python Software Foundation. Linux® es una marca registrada de Linus Torvalds. Ubuntu® es una marca registrada de Canonical Limited. Google® es una marca registrada de Google Inc. Microsoft® y Windows® son marcas registradas de Microsoft Corporation. Red Hat® es una marca comercial registradas de Red Hat, Inc. UNIX® es una marca comercial registrada de The Open Group. Apple® y Mac OS® son marcas comerciales de Apple Inc.

Versión	Autor/es	Fecha	Observaciones
1.0	Marcelo Horacio Fortino	2021/Marzo	Curso Python
1.1	Marcelo Horacio Fortino	2021/Junio	Convertido a markdown - ipynb
1.2	Marcelo Horacio Fortino	2021/Agosto	Actualizados contenidos
1.3	Marcelo Horacio Fortino	2021/Octubre	Agregado Flask microframework
1.4	Marcelo Horacio Fortino	2021/Noviembre	Agregado Pandas - datascience
1.5	Marcelo Horacio Fortino	2021/Diciembre	Agregado Devops - Ansible
2.0	Marcelo Horacio Fortino	2022/Abril	Nueva estructura: core / module
2.1	Marcelo Horacio Fortino	2022/Junio	Módulo apuntes intermedio
2.2	Marcelo Horacio Fortino	2022/Agosto	Actualizado temario y ejercicios

Versión	Autor/es	Fecha	Observaciones
2.3	Marcelo Horacio Fortino	2022/Octubre	Actualizado despliegue a render.com
2.4	Marcelo Horacio Fortino	2023/Noviembre	Aplicaciones de escritorio con python
2.5	Marcelo Horacio Fortino	2026/Abril	Aplicaciones de ML e IA para python

Esta obra se distribuye con la esperanza de que sea útil, pero SIN NINGUNA GARANTÍA, incluso sin la garantía MERCANTIL implícita o sin garantizar la CONVENIENCIA PARA UN PROPÓSITO PARTICULAR. El autor no asume ninguna responsabilidad si el lector hace un mal uso de la misma.

- Estos apuntes se basan en:
 - La documentación oficial de Python, <https://docs.python.org/es/3/tutorial/index.html>,
 - La bibliografía presentada al final de este documento, y
 - Documentación propia recogida a lo largo de los años de diversas fuentes.

1.2 Objetivo del curso

- Objetivo general del curso:
- Aprender a usar Python para crear scripts y programas simples plenamente funcionales, desarrollar aplicaciones web, realizar análisis de datos, y utilizar bibliotecas de inteligencia artificial (AI) para aprendizaje automático (ML).
- Objetivos específicos:
 - Reconocer las características principales de Python y su utilidad práctica.
 - Identificar tipos de datos (simples y compuestos) y operadores.
 - Aplicar variables y estructuras de control de flujo.
 - Construir funciones y clases (POO).
 - Clasificar los módulos y paquetes por sus funcionalidades y objetivos.
 - Comparar el lenguaje con otros similares de scripts, procedimentales y orientados a objetos.
 - Probar bibliotecas para conexiones REST a aplicaciones web y bases de datos.
 - Resolver problemas y errores en el código fuente proponiendo soluciones alternativas (refactoring).
 - Utilizar la biblioteca pandas junto con matplotlib y numpy para realizar análisis estadísticos de datos y gráficos.
 - Descubrir las herramientas en python de ML y DL (Deep Learning) para la inteligencia artificial.
- Como resultado práctico al final del curso cada estudiante habrá creado una aplicación web utilizando el microframework de Python Flask.

1.3 Temario

- Introducción, instalación y compilación

- Datos, expresiones y sentencias
- Variables y funciones, control de flujo
- Clases y objetos, herencia, polimorfismo
- Entradas y salidas con Python
- Gestión de módulos, paquetes y bibliotecas
- Servicios y programas en red, REST API
- Desarrollo de aplicaciones web con Flask
- Análisis de datos con pandas, matplotlib y numpy
- Módulos opcionales
 - DevOps con Ansible
 - Plotting con Python
 - SDK de GCP para Python
 - SQL ejemplos con pandas
 - Docker y kubernetes con Python
 - Machine Learning con Python
 - Aplicaciones de escritorio con python

1.4 Bibliografía

- Downey, A., Elkner, J., Meyers, C. (2015). Aprenda a Pensar Como un Programador con Python.
Recuperado de <https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>
- Kent D. Lee. (2014). Python, Programming Fundamentals Second Edition.
- Marzal Varó, A., Gracia Luengo, I., García Sevilla, Pedro. (2014). Introducción a la programación con Python 3.
Recuperado de <http://repositori.uji.es/xmlui/handle/10234/102653>
- Miller, B., Ranum, D. Solución de problemas con algoritmos y estructuras de datos usando Python. Traducido por Mauricio Orozco-Alzate, Universidad Nacional de Colombia - Sede Manizales.
Recuperado de <https://runestone.academy/ns/books/published/pythoned/index.html?mode=browsing>
- Shaw, Z. A. (2016). Learn Python 3 the Hard Way.
Recuperado de <https://learnpythonthehardway.org/>
- Van Rossum, G. and the Python development team. (2020). Documentación de Python en español.
Recuperado de https://python-docs-es.readthedocs.io/_/downloads/es/pdf/pdf/
- McGregor, S. E. (2022). Practical Python data wrangling and data quality. O'Reilly Media.
- Chollet, F. (2017). Deep learning with Python. Manning Publications.
- Russell, S., & Norvig, P. (2019). Artificial intelligence: A modern approach. Pearson Higher Education.
- D2L.ai: Interactive Deep Learning Book with Multi-Framework Code, Math, and Discussions.
Recuperado de <https://d2l.ai/>

2 Aprendizaje automático

- Si bien se viene estudiando la forma de hacer que las máquinas puedan aprender por si solas desde 1970 aproximadamente, no fue hasta 1983 con la publicación del libro *Machine learning: The AI Approach* (Ryszard S. Michalski, Tom M. Mitchell, Jaime Carbonell) que la disciplina de aprendizaje automático (*Machine Learning - ML*) tomó impulso.
- Ésta fue luego sistematizada en otro libro seminal escrito por T. Mitchell: *Machine Learning* (1997).
- El aprendizaje automático (*Machine Learning - ML*) incluye varias técnicas que le permiten a los algoritmos aprender patrones de datos y luego hacer predicciones probabilísticas para su posterior análisis y toma de decisiones.
- Incluye el aprendizaje supervisado (*supervised learning*), donde los modelos se entrenan en datos etiquetados; aprendizaje no supervisado (*unsupervised learning*), donde los patrones se reconocen mediante datos no etiquetados; y aprendizaje de refuerzo (*reinforcement learning*), en el cual los modelos aprenden a tomar decisiones basados en recompensas o castigos.
- Fuente: KUBAT, Miroslav. (2017). An Introduction to Machine Learning. Second Edition Switzerland: Springer.
- Para una introducción básica a ML: <https://www.databricks.com/blog/machine-learning>.

2.1 Bibliotecas de python

- Entre las herramientas más populares de python para aprendizaje automático (ML), aprendizaje profundo (DL), LLM / GenAI, y canalizaciones / automatización se encuentran:
- Aprendizaje automático (ML)
 - Scikit-learn <https://scikit-learn.org/stable/>, construida sobre NumPy, SciPy, y matplotlib.
 - XGBoost, LightGBM, y CatBoost <https://pythondatascience.com/article/gradient-boosting-python-xgboost-lightgbm-catboost-2026>.
- Aprendizaje profundo (DL)
 - TensorFlow <https://www.tensorflow.org/about>.
 - PyTorch <https://pytorch.org/>.
 - Keras < <https://keras.io/> >.
 - La más reciente JAX <https://github.com/google/jax>.
 - Transformers (Hugging Face) <https://huggingface.co/docs/transformers/index> es el estándar de facto para modelos de aprendizaje profundo preentrenados.
- LLMs y GenAI
 - Conectan LLMs a fuentes de datos (PDFs, BBDD relacionales, etc.) y construyen agentes de IA.
 - LangChain <https://www.langchain.com/>.
 - LlamaIndex <https://www.llamaindex.ai/>.
- Canalizaciones y automatización
 - MLflow <https://mlflow.org/>.
 - Kedro <https://kedro.org/>.
 - Prefect <https://www.prefect.io/>.

- Airflow <https://airflow.apache.org/>.

2.2 TensorFlow

- El aprendizaje automático es la práctica de ayudar al software a realizar una tarea sin programación o reglas explícitas.
- En la programación tradicional de computadoras, un programador especifica las reglas que debe usar la computadora.
- Sin embargo, ML requiere una mentalidad diferente:
 - El aprendizaje automático del mundo real se centra mucho más en el análisis de datos que en la codificación.
 - Los programadores proporcionan un conjunto de ejemplos y la computadora aprende patrones a partir de los datos.
 - Se puede entender el aprendizaje automático como “programación con datos”.
- Extraído de TensorFlow: <https://www.tensorflow.org/about>.
- Los tensores son matrices multidimensionales con un tipo uniforme denominado (*dtype*), en NumPy los tensores son similares a las matrices (*np.arrays*).
- Todos los tensores son inmutables como los números y las cadenas de Python. Nunca se puede actualizar el contenido de un tensor, solo se puede crear uno nuevo.
 - Extraído de <https://www.tensorflow.org/guide/tensor?hl=es>.

2.3 Pasos en ML

- Paso 1: recopilar datos.
- Paso 2: Explorar los datos.
- Paso 2.5: Elegir un modelo.
- Paso 3: Preparar los datos.
- Paso 4: construir, entrenar y evaluar el modelo.
- Paso 5: Sintonizar los hiperparámetros.
- Paso 6: Implementar el modelo.
- Extraído de Maching Learning:
 - <https://developers.google.com/machine-learning/guides/text-classification/>.

Actividad ML:

Realizar la guía de clasificación de texto de Google for developers <https://developers.google.com/machine-learning/guides/text-classification?hl=es-419>.

2.4 Redes neuronales

- Es un tipo de modelo que se puede entrenar para reconocer patrones.
- Está compuesto por capas, incluidas las de entrada y salida, y al menos una capa oculta.
- Las neuronas de cada capa aprenden representaciones cada vez más abstractas de los datos.
- Extraído de TensorFlow: <https://www.tensorflow.org/about>.
- Se suelen dividir en dos tipos: las redes neuronales convolucionales (*Convolutional Neural Networks* - *CNN*), se utilizan para analizar imágenes, mientras que las redes neuronales

recurrentes (*Recurrent Neural Networks* - *RNN*) se utilizan para datos de secuencias.

2.5 Entrenar una red neuronal

- Las redes neuronales se entrenan por descenso de gradiente.
- Los pesos en cada capa comienzan con valores aleatorios, y estos se mejoran iterativamente con el tiempo para hacer que la red sea más precisa.
- Se usa una función de pérdida para cuantificar qué tan imprecisa es la red, y se usa un procedimiento llamado retropropagación para determinar si se debe aumentar o disminuir cada peso para reducir la pérdida.

2.6 PyTorch

- PyTorch fue originalmente desarrollada por Meta y actualmente se gestiona mediante la PyTorch Foundation, una comunidad de desarrolladores, investigadores y pioneros de la industria de IA que colaboran para llevar adelante los proyectos de la misma.
- Fundación PyTorch: <https://pytorch.org/>.
- PyTorch es un paquete de Python que provee dos funcionalidades de alto nivel:
 - Un tensor computacional (similar a *NumPy*) con una fuerte aceleración para las GPUs.
 - Redes neuronales profundas construidas sobre un sistema de autograd (*tape-based autograd system*) basado en cinta.
- Se pueden reutilizar también, de ser necesario, los paquetes de python tipo *NumPy*, *SciPy* y *Cython* para extender PyTorch.
- Fuente: <https://github.com/pytorch/pytorch>.
- Tutorial básico: https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html.

2.7 Keras

- Keras 3 es una API multi-framework de aprendizaje profundo.
- Puede ser usada para desarrollar componentes modulares compatibles con otros marcos tipo JAX, TensorFlow, o PyTorch.
- Es simple, flexible y poderosa, siendo utilizada en organizaciones tipo la NASA o Alphabet.
- Fuente: https://keras.io/getting_started/about/.
- Guías de uso: <https://keras.io/guides/>.

2.8 JAX

- Jax es una biblioteca de Python diseñada para realizar investigación de alto desempeño en aprendizaje automático.
- Es una biblioteca computacional numérica similar a Numpy pero con algunas mejoras claves.
- Ha sido desarrollada internamente en Google, donde es utilizada por los equipos de Deepmind.
- Fuentes: <https://github.com/jax-ml/jax>, <https://docs.jax.dev/en/latest/>.
- El Stack de JAX AI es una colección de bibliotecas seleccionadas por investigadores e ingenieros de Google y otras organizaciones, útiles para implementar y desplegar modelos de las herramientas de IA generativa (GenAI) tipo Imagen o Gemini, entre otras.

- Fuente: <https://docs.jaxstack.ai/en/latest/index.html>.

2.9 Apache Spark MLlib

- MLlib es la biblioteca escalable de aprendizaje automático (ML) de Apache Spark.
- Su objetivo es hacer que el aprendizaje automático práctico sea escalable y fácil.
 - Fuente: <https://spark.apache.org/mllib/>.
- Las utilidades de flujo de trabajo de ML incluyen:
 - Algoritmos de ML: algoritmos de aprendizaje comunes como clasificación, regresión, agrupamiento y filtrado colaborativo.
 - Caracterización: extracción de características, transformación, reducción de dimensionalidad y selección.
 - * Fuente: <https://spark.apache.org/docs/latest/ml-guide.html>.
- Canalizaciones:
 - Incluye herramientas para construir, evaluar y ajustar canalizaciones de ML.
- Persistencia:
 - Guarda y carga algoritmos, modelos y canalizaciones.
- Utilidades:
 - Disponibles para álgebra lineal, estadística, manejo de datos, etc.

2.10 PySpark

- PySpark es la API de Python para Apache Spark utilizada para procesar datos a gran escala y en tiempo real en un entorno distribuido.
- Proporciona además una consola PySpark para analizar los datos de forma interactiva, combinando la facilidad de aprendizaje y uso de Python con la potencia de Apache Spark.
- Es compatible con todas las características de Spark, como Spark SQL, *DataFrames*, *Structured Streaming*, *Machine Learning (MLlib)*, *Pipelines* y *Spark Core*.
- Fuente: <https://spark.apache.org/docs/latest/api/python/index.html>.
- *Cheat sheet*: <https://www.sparkplayground.com/pyspark-cheat-sheet>.

3 Tutorial Spark MLLib

- Para realizar este tutorial abrimos un cuaderno de Jupyter <https://jupyter.org/> en nuestro servidor local, utilizar Colaboratory: <https://colab.research.google.com>, o lanzar un contenedor con Spark.
- Para esto último, los pasos son los siguientes:

```
[ ]: docker run -d -v ${PWD}:/home/jovyan/work -p 8888:8888 -p 4040:4040 -p 4041:4041 --name pyspark jupyter/pyspark-notebook
```

- Abrimos en el navegador la URL <http://127.0.0.1:8888>. La clave se puede obtener en el log del contenedor.
- Una vez dentro de *Jupyter Lab* abrimos un *notebook* con el kernel de python.
- Para *Google Colab* los pasos son los siguientes:

- Se abre un nuevo cuaderno *jupyter* y se instala *Spark* y las dependencias y bibliotecas necesarias:

```
[ ]: # Instalación
!apt update
```

```
[ ]: !apt upgrade
```

```
[ ]: !apt install default-jre
```

```
[ ]: !apt install default-jdk
```

```
[ ]: !apt install scala
```

```
[ ]: # Comprobación
!java -version
!javac -version
!scala -version
!readlink -f $(which java)
```

```
[ ]: import os
# Crear el directorio para Spark
!mkdir -p /content/spark-3.5.1-bin-hadoop3
# Descargar Spark
!wget -q https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.
↪1-bin-hadoop3.tgz -P /content
```

```
[ ]: !ls
```

```
[ ]: !tar xvf spark-3.5.1-bin-hadoop3.tgz
```

```
[ ]: !pip install pyspark==3.5.1
```

- Se agregan las rutas de *java* y *spark* al ambiente:

```
[ ]: import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64"
os.environ["SPARK_HOME"] = "/content/spark-3.5.1-bin-hadoop3"
```

3.1 Crear sesión en Spark

- En el cuaderno instalamos y configuramos la biblioteca de Python *findspark*:

```
[2]: pip install -q findspark
```

```
[ ]: import findspark
findspark.init()
from pyspark.sql import SparkSession
```



```
spark = SparkSession.builder.master("local[*]").getOrCreate()
```

```
[ ]: df = spark.createDataFrame([{"Hola": "Mundo"} for x in range(1000)])  
df.show(3, False)
```

- A continuación, la adaptación y traducción del tutorial de MA Raza, Ph.D.
- Disponible en <https://towardsdatascience.com/machine-learning-with-spark-f1dbc1363986>.
- Importamos la biblioteca *pyspark*:

```
[ ]: import pyspark  
print(pyspark.__version__)
```

3.2 Normalizar datos

- Utilizamos la clase *MinMaxScaler* en ML para normalizar datos numéricos.
- Este *transformer* escala los datos a un rango específico, generalmente entre 0 y 1.
- Es un paso de preprocesado común en aprendizaje automático.

```
[ ]: from pyspark.ml.feature import MinMaxScaler  
from pyspark.ml.linalg import Vectors
```

- En *PySpark's MLlib (Machine Learning Library)* las características o funcionalidades son generalmente representadas como *Vectors* densos o escasos.
- En este caso se crean vectores densos para estas características.
- Fuente: <https://krunkanojiya.com/blog/dense-vs-sparse-vectors>.

```
[ ]: features_df = spark.createDataFrame([  
    (1, Vectors.dense([10.0,10000.0,1.0])),  
    (2, Vectors.dense([20.0,40000.0,2.0])),  
    (3, Vectors.dense([30.0,50000.0,3.0])),  
],["id", "features"] )
```

```
[ ]: features_df.show()
```

- Aplicamos la transformación de la biblioteca *MinMaxScaler*:

```
[ ]: features_scaler = MinMaxScaler(inputCol = "features", outputCol = "sfeatures")  
smodel = features_scaler.fit(features_df)  
sfeatures_df = smodel.transform(features_df)
```

```
[ ]: sfeatures_df.show()
```

- Utilizamos la clase *StandardScaler* para estandarizar datos en ML.
- *StandardScaler* es un *transformer* que estandariza las características eliminando la media y escalándolas a varianza unitaria.
- Los escala entre -1 y 1.

```
[ ]: from pyspark.ml.feature import StandardScaler
from pyspark.ml.linalg import Vectors
```

```
[ ]: features_df = spark.createDataFrame([
    (1, Vectors.dense([10.0,10000.0,1.0])),
    (2, Vectors.dense([20.0,40000.0,2.0])),
    (3, Vectors.dense([30.0,50000.0,3.0])),
],["id", "features"] )
```

```
[ ]: features_stand_scaler = StandardScaler(inputCol = "features", outputCol =
    ↪ "sfeatures", withStd=True, withMean=True)
stmodel = features_stand_scaler.fit(features_df)
stand_sfeatures_df = stmodel.transform(features_df)
```

```
[ ]: stand_sfeatures_df.show(20, False)
```

- La clase *Bucketizer* transforma los datos en varias frecuencias o *buckets*.
- Por ejemplo separando valores en categorías según umbrales predefinidos.
- Es un paso de preprocesamiento útil para ciertos algoritmos de aprendizaje automático.

```
[ ]: from pyspark.ml.feature import Bucketizer
from pyspark.ml.linalg import Vectors
```

```
[ ]: splits = [-float("inf"), -10, 0.0, 10, float("inf")]
b_data = [(-800.0,), (-10.5,), (-1.7,), (0.0,), (8.2,), (90.1,)]
b_df = spark.createDataFrame(b_data, ["features"])
```

```
[ ]: b_df.show()
```

```
[ ]: bucketizer = Bucketizer(splits=splits, inputCol= "features",
    ↪ outputCol="bfeatures")
bucketed_df = bucketizer.transform(b_df)
```

```
[ ]: bucketed_df.show()
```

3.3 NLP - Natural Language Processing

- Uno de los primeros pasos en NLP (*Natural Language Processing*) es convertir el texto en *tokens* o palabras *tokenizadas*:

```
[ ]: from pyspark.ml.feature import Tokenizer
```

```
[ ]: oraciones_df = spark.createDataFrame([
    (1, "Introducción a sparkMlib"),
    (2, "Mlib incluye bibliotecas para clasificación y regresión"),
    (3, "También incluye soporte a datapipe lines"),
```

```
], ["id", "oraciones"])
```

```
[ ]: oraciones_df.show()
```

- Para reflejar la importancia de una palabra en un texto utilizamos *Term frequency-inverse document frequency (TF-IDF)*:

```
[ ]: sent_token = Tokenizer(inputCol = "oraciones", outputCol = "palabras")
sent_tokenized_df = sent_token.transform(oraciones_df)
```

```
[ ]: sent_tokenized_df.take(10)
```

```
[ ]: from pyspark.ml.feature import HashingTF, IDF
hashingTF = HashingTF(inputCol = "palabras", outputCol = "rawfeatures",
    ↪ numFeatures = 20)
sent_fhTF_df = hashingTF.transform(sent_tokenized_df)
```

```
[ ]: sent_fhTF_df.take(1)
```

```
[ ]: idf = IDF(inputCol = "rawfeatures", outputCol = "idffeatures")
idfModel = idf.fit(sent_fhTF_df)
tfidf_df = idfModel.transform(sent_fhTF_df)
```

```
[ ]: tfidf_df.take(1)
```

3.4 Agrupamiento - Clustering

- Para agrupar datos en un razonable grupo de frecuencias se puede utilizar como técnica el llamado *clustering*:

```
[ ]: from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans, BisectingKMeans
import glob
```

```
[37]: # Descarga el dataset
!wget -q 'https://raw.githubusercontent.com/amjadraza/blogs-data/master/
    ↪ spark_ml/clustering_dataset.csv'
```

```
[ ]: clustering_file_name = 'clustering_dataset.csv'
import pandas as pd
```

```
[ ]: cluster_df = spark.read.csv(clustering_file_name, header=True, inferSchema=True)
```

- Convierte los datos tabulares a un formato vectorizado.

```
[ ]: vectorAssembler = VectorAssembler(inputCols = ['col1', 'col2', 'col3'],  
    ↪outputCol = "features")  
vcluster_df = vectorAssembler.transform(cluster_df)
```

```
[ ]: vcluster_df.show(10)
```

- Se aplica el algoritmo de *k-means*.
- *KMeans* agrupa puntos de datos sin etiquetar en un número predefinido de clústeres, denotado por *k*.
- Luego divide *n* observaciones en *k* clústeres, donde cada observación pertenece al clúster con la media (centroide) más cercana.

```
[ ]: kmeans = KMeans().setK(3)  
kmeans = kmeans.setSeed(1)  
kmodel = kmeans.fit(vcluster_df)
```

```
[ ]: centers = kmodel.clusterCenters()  
print("The location of centers: {}".format(centers))
```

- Otro algoritmo de *clustering* implementado en MLlib es el llamado *Bisecting K-Means*.

```
[ ]: bkmeans = BisectingKMeans().setK(3)  
bkmeans = bkmeans.setSeed(1)  
bkmodel = bkmeans.fit(vcluster_df)  
bkcneters = bkmodel.clusterCenters()
```

```
[ ]: bkcneters
```

3.5 Clasificación utilizando PySpark

- En estos ejemplos se utilizará el dataset *iris* de UCI <https://archive.ics.uci.edu/ml/datasets/iris>.
- Obtenemos el fichero desde Internet con *wget* e importamos la biblioteca *pandas*:

```
[ ]: !wget -q "https://raw.githubusercontent.com/amjadraza/blogs-data/master/  
    ↪spark_ml/iris.csv"  
import pandas as pd
```

- Se crea el *dataframe* con el fichero descargado:

```
[ ]: df = pd.read_csv("iris.csv", header=None)
```

```
[ ]: df.head()
```

- Para realizar modelos ML, aplicamos el paso de preprocesamiento en nuestros datos de entrada:

```
[ ]: from pyspark.sql.functions import *
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.feature import StringIndexer
```

```
[ ]: df_iris = pd.read_csv("iris.csv", header=None)
iris_df = spark.createDataFrame(df_iris)
```

```
[ ]: iris_df.show(5, False)
```

- Renombramos las columnas:

```
[ ]: iris_df = iris_df.select(col("0").alias("sepal_length"),
                             col("1").alias("sepal_width"),
                             col("2").alias("petal_length"),
                             col("3").alias("petal_width"),
                             col("4").alias("species"),
                             )
```

- Convertimos las columnas en características (*features*):

```
[ ]: vectorAssembler = VectorAssembler(inputCols = ["sepal_length", "sepal_width",
↪ "petal_length", "petal_width"],
                                     outputCol = "features")
viris_df = vectorAssembler.transform(iris_df)
```

```
[ ]: viris_df.show(5, False)
```

```
[ ]: indexer = StringIndexer(inputCol="species", outputCol = "label")
iviris_df = indexer.fit(viris_df).transform(viris_df)
```

```
[ ]: iviris_df.show(5, False)
```

3.6 La clasificación Naive Bayes

- *Naive Bayes* es una familia de clasificadores probabilísticos simples que aplican el teorema de Bayes con supuestos de independencia ingenuos (*naive*) entre las características.
- Se utilizan comúnmente para tareas como la clasificación de texto y la detección de spam.
- *MulticlassClassificationEvaluator* se utiliza para valorar el rendimiento de los modelos de ML en tareas de clasificación multiclase.
- Puede calcular diversas métricas como la puntuación F1, la precisión, la exhaustividad y la exactitud, que son fundamentales para entender el rendimiento de un modelo de clasificación.

```
[ ]: from pyspark.ml.classification import NaiveBayes
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

- Creamos los *splits* de entrenamiento y test:

```
[ ]: splits = iviris_df.randomSplit([0.6,0.4], 1)
train_df = splits[0]
test_df = splits[1]
```

- Aplicamos la clasificación Naive bayes:

```
[ ]: nb = NaiveBayes(modelType="multinomial")
nbmodel = nb.fit(train_df)
predictions_df = nbmodel.transform(test_df)
```

```
[ ]: predictions_df.show(1, False)
```

- Evaluamos el clasificador entrenado:

```
[ ]: evaluator = MulticlassClassificationEvaluator(labelCol="label",
                                                    predictionCol="prediction",
                                                    metricName="accuracy")

nbaccuracy = evaluator.evaluate(predictions_df)
nbaccuracy
```

3.7 Clasificación de Multilayer Perceptron

- La clase *MultilayerPerceptronClassifier* provee una implementación de una red neuronal (*artificial neural network* - ANN) para tareas de clasificación.
- Es un tipo de red neuronal con múltiples capas ocultas, capaz de aprender relaciones no lineales en los datos.
- Se pueden configurar parámetros como el número de capas, el número de neuronas en cada capa, y las funciones de activación.

```
[ ]: from pyspark.ml.classification import MultilayerPerceptronClassifier
```

```
[ ]: layers = [4,5,5,3]
mlp = MultilayerPerceptronClassifier(layers = layers, seed=1)
mlp_model = mlp.fit(train_df)
mlp_predictions = mlp_model.transform(test_df)
```

```
[ ]: mlp_evaluator = MulticlassClassificationEvaluator(labelCol="label",
                                                        predictionCol="prediction",
                                                        metricName="accuracy")

mlp_accuracy = mlp_evaluator.evaluate(mlp_predictions)
mlp_accuracy
```

3.8 Clasificación con árboles de decisión

- La clase *DecisionTreeClassifier* proporciona una implementación de un algoritmo de árbol de decisión para tareas de clasificación dentro de la biblioteca MLlib de PySpark.
- Los árboles de decisión son métodos de aprendizaje supervisado no paramétricos que se utilizan tanto para clasificación como para regresión.

- Funcionan creando un modelo que predice el valor de una variable objetivo aprendiendo reglas de decisión simples inferidas a partir de las características de los datos.
- Para la clasificación, el objetivo es dividir los datos en subconjuntos que contengan instancias con etiquetas de clase similares.

```
[ ]: from pyspark.ml.classification import DecisionTreeClassifier
```

```
[ ]: dt = DecisionTreeClassifier(labelCol="label", featuresCol="features")
dt_model = dt.fit(train_df)
dt_predictions = dt_model.transform(test_df)
```

```
[ ]: dt_evaluator = MulticlassClassificationEvaluator(labelCol="label",
                                                    predictionCol="prediction",
                                                    metricName="accuracy")

dt_accuracy = dt_evaluator.evaluate(dt_predictions)
dt_accuracy
```

- Se pueden encontrar otros algoritmos de clasificación de la biblioteca Spark MLlib en: <https://spark.apache.org/docs/latest/ml-classification-regression.html#classification>.

3.9 Modelos de Regresión

- Los modelos de regresión son útiles para predecir valores futuros utilizando datos del pasado.

```
[ ]: from pyspark.ml.regression import LinearRegression
from pyspark.ml.feature import VectorAssembler
```

- En este ejemplo se utiliza el conjunto de datos (*dataset*) de *Combined Cycle Power Plant* para predecir el consumo eléctrico (*net hourly electrical output -EP*).
- Obtenemos el fichero desde Internet con *wget* e importamos la biblioteca *pandas*:

```
[7]: !wget -q "https://raw.githubusercontent.com/amjadraza/blogs-data/master/
↳spark_ml/ccpp.csv"
import pandas as pd
```

```
[ ]: df_ccpp = pd.read_csv("ccpp.csv")
```

```
[ ]: pp_df = spark.createDataFrame(df_ccpp)
pp_df.show(2, False)
```

- Se crea la columna de características utilizando la clase *VectorAssembler*:

```
[ ]: vectorAssembler = VectorAssembler(inputCols=["AT", "V", "AP", "RH"], outputCol=
↳ "features")
vpp_df = vectorAssembler.transform(pp_df)
```

```
[ ]: vpp_df.show(2, False)
```

- Se establece la regresión lineal (*Linear Regression*):

```
[ ]: lr = LinearRegression(featuresCol="features", labelCol="PE")
lr_model = lr.fit(vpp_df)
```

```
[ ]: lr_model.coefficients
lr_model.intercept
lr_model.summary.rootMeanSquaredError
```

- Un ejemplo utilizando árboles de regresión (*Decision Tree Regression*):

```
[ ]: from pyspark.ml.regression import DecisionTreeRegressor
from pyspark.ml.evaluation import RegressionEvaluator
```

```
[ ]: vpp_df.show(2, False)
```

- Definir, entrenar y testar los *splits* de datos:

```
[ ]: splits = vpp_df.randomSplit([0.7,0.3])
train_df = splits[0]
test_df = splits[1]
```

- Se define el modelo del árbol de regresión:

```
[ ]: dt = DecisionTreeRegressor(featuresCol="features", labelCol="PE")
dt_model = dt.fit(train_df)
dt_predictions = dt_model.transform(test_df)
```

```
[ ]: dt_predictions.show(1, False)
```

- Se evalúa el modelo:

```
[ ]: dt_evaluator = RegressionEvaluator(labelCol="PE", predictionCol="prediction",
metricName="rmse")
dt_rmse = dt_evaluator.evaluate(dt_predictions)
```

```
[ ]: print("The RMSE of Decision Tree regression Model is {}".format(dt_rmse))
```

- La clase *GBTRegressor* o regresor de árbol potenciado por gradiente (*Gradient-Boosted Tree Regressor*) es un algoritmo de ML utilizado para tareas de regresión.
- Construye un conjunto de árboles de decisión de forma secuencial, donde cada nuevo árbol corrige los errores de los anteriores, lo que da como resultado un modelo predictivo más preciso y robusto.
- Es utilizada con frecuencia para tareas en las que el objetivo es predecir un valor numérico continuo. Un ejemplo con *Gradient Boosting* puede ser:

```
[ ]: from pyspark.ml.regression import GBTRegressor
```



```
[ ]: gbt = GBRegressor(featuresCol="features", labelCol="PE")
      gbt_model = gbt.fit(train_df)
      gbt_predictions = gbt_model.transform(test_df)

[ ]: gbt_evaluator = RegressionEvaluator(labelCol="PE", predictionCol="prediction",
      ↪metricName="rmse")
      gbt_rmse = gbt_evaluator.evaluate(gbt_predictions)

[ ]: print("The RMSE of GBT Tree regression Model is {}".format(gbt_rmse))
```